PROJECT DOCUMENTATION

How We Built Dr. Monisha's Website

A plain-English walkthrough of every step taken — from blank page to a fully live, installable web app with real-time bookings and push notifications.

8	4	2	PWA
CHAPTERS	API ENDPOINTS	CORE PAGES	INSTALLABLE APP

CONTENTS

What's Inside

01	The Goal — What We Were Building	Scope & vision
02	The Public Website (index.html)	Design, sections, and the booking form
03	The Database — Supabase	Where bookings are stored and how security works
04	The Admin Dashboard (admin.html)	Managing bookings, WhatsApp, and status updates
05	The Backend — Vercel API	Serverless functions that power everything
06	Push Notifications — Firebase	Getting alerted the moment a booking arrives
07	PWA — Installing the App on a Phone	Service worker, manifest, and offline support
08	Deployment — Going Live	Git, Vercel, and environment variables

TECH STACK AT A GLANCE

HTML / CSS / JavaScript · Tailwind CSS · Supabase (Database) · Firebase (Notifications) · Vercel (Hosting) · PWA / Service Worker · WhatsApp Integration

CHAPTER 01

The Goal — What We Were Building

Before writing a single line of code, we needed to clearly understand what the website had to do and who would use it.

Two Types of Users

This project has two completely different audiences, each with their own needs:

👩‍⚕️ Patients (Public)

People who visit the website, learn about Dr. Monisha's services, and want to book a consultation. They need a clean, trustworthy, and easy-to-use website.

👩‍💻 Dr. Monisha (Admin)

The doctor who needs to see incoming booking requests, confirm or reject them, contact patients via WhatsApp, and get notified instantly when someone books.

What the Website Needed to Do

1. **Present the practice professionally**
   Show Dr. Monisha's services, credentials, and contact info in a clean, modern design that builds trust.
2. **Accept booking requests online**
   Let patients fill out a form with their name, phone, preferred date/time, and consultation type — without needing to call.
3. **Notify Dr. Monisha instantly**
   Send a push notification the moment a new booking arrives, even if the browser is closed.
4. **Let the doctor manage bookings**
   A private admin panel to confirm, reject, search, filter, and delete bookings — and message patients on WhatsApp.
5. **Work like a mobile app**
   The admin dashboard can be installed on a phone's home screen and work offline — no app store required.

WHY A WEBSITE INSTEAD OF AN APP?

Publishing an app on the App Store or Play Store takes weeks, costs money, and requires approval. A Progressive Web App (PWA) website can be installed directly from the browser in seconds — and it looks and feels just like a native app.

CHAPTER 02

The Public Website (index.html)

The face of the practice. Everything a patient sees when they visit the URL. Built as a single HTML file with all styles embedded inside it.

Why One Single File?

We put the entire website in one `index.html` file. This keeps things simple — no build tools, no complex setup. The browser just opens one file and everything works. For a marketing website of this size, a single file is the right choice.

The Sections We Built

1. **Hero Section — The First Thing You See**

   A bold headline, Dr. Monisha's photo, a tagline, and a "Book a Consultation" button. This section needs to immediately communicate who she is and what she does. We used large typography, soft background gradients, and her real photo.

2. **About Section — Building Trust**

   A short bio introducing Dr. Monisha. Trust is everything in healthcare — patients need to feel safe before booking. This section humanises her practice.

3. **Services Section — What She Offers**

   Six cards listing the mental health services available: anxiety, depression, counselling, etc. Each card has an icon, title, and brief description. Laid out in a responsive grid that collapses to one column on phones.

4. **Testimonials — Social Proof**

   Patient quotes. People trust other people's experiences. Testimonials reduce hesitation and make new patients feel comfortable reaching out.

5. **Booking Modal — The Core Action**

   When someone clicks "Book Now", a dialog pops up with a form. They fill in their name, phone number, email, preferred date, time, and whether they want an online or offline consultation. When they submit, the data is saved to the database.

6. **Footer — Contact Info**

   Links, social media, phone number, and location. Patients who scroll all the way down are very interested — the footer captures them.

How It Was Styled

We used **Tailwind CSS** — a utility-first CSS framework loaded from a CDN link. Instead of writing custom CSS from scratch, Tailwind gives us thousands of tiny pre-built classes like `text-lg`, `p-4`, or `rounded-xl` that we compose together directly in the HTML.

Brand Colors

Navy **#1a2870** for headings and buttons. Rose/blush **#c17b6e** for accents. Soft cream **#fdf5f2** for backgrounds. These were chosen to feel calm, professional, and trustworthy for a mental health practice.

Typography

Playfair Display (a serif font) for headings — elegant and authoritative. **DM Sans** for body text — clean, modern, and easy to read on screens.

MOBILE-FIRST DESIGN

We designed for mobile screens first, then scaled up for desktop. This is the correct approach because most patients will visit the site on their phones. Tailwind's `md:` and `lg:` prefixes let us write rules that only activate on larger screens.

CHAPTER 03

The Database — Supabase

When a patient submits a booking form, that data needs to be stored somewhere safely so Dr. Monisha can access it later. That's the database's job.

What is Supabase?

Supabase is a ready-made backend service. Think of it as a spreadsheet in the cloud that your code can read from and write to. It uses a real database (PostgreSQL) under the hood, but provides a simple JavaScript SDK so you don't need to be a database expert to use it.

We chose Supabase because it has a generous free tier, is fast to set up, and handles all the server infrastructure — we don't need to manage any servers ourselves.

The Bookings Table

We created a table called `bookings` in Supabase. A table is like a spreadsheet — rows are individual bookings, columns are the pieces of information for each booking.

COLUMN	WHAT IT STORES	EXAMPLE
`id`	Unique ID for each booking	auto-generated
`name`	Patient's full name	Priya Sharma
`phone`	Patient's phone number	9876543210
`email`	Patient's email address	priya@email.com
`date`	Requested appointment date	2026-03-15
`time`	Requested appointment time	10:30 AM
`type`	Online or offline consultation	online
`status`	Current status of the booking	pending / confirmed / rejected
`submitted_at`	When the form was submitted	auto-timestamp

Security — Row Level Security (RLS)

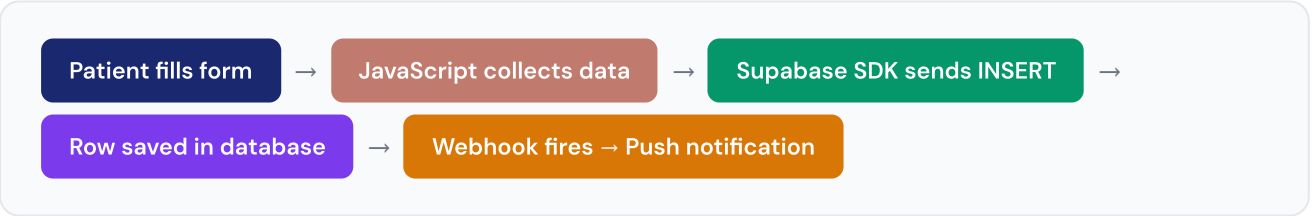
Supabase has a security feature called **Row Level Security (RLS)**. By default, the database is completely locked — nobody can read or write anything. You have to explicitly create "policies" that allow specific actions.

We set up four policies on the bookings table:

- ✓ **INSERT policy — Anyone can submit a booking**
  Allows the booking form on the public website to add new rows to the table.
- ✓ **SELECT policy — The admin can read bookings**
  Allows the dashboard to fetch and display all bookings.
- ✓ **UPDATE policy — The admin can change status**
  Allows the dashboard to mark a booking as confirmed, rejected, or booked.
- ✓ **DELETE policy — The admin can remove bookings**
  Allows the dashboard to permanently delete a booking from the database.

> **WHY THIS MATTERS**
> Without these policies, operations silently fail — buttons appear to work but nothing actually changes in the database. If status updates or deletes ever stop working, checking these RLS policies is always the first step.

How the Booking Form Connects to Supabase



The Supabase JavaScript SDK is loaded via a CDN link in the HTML. We initialize it with the project URL and an API key, then call `supabase.from('bookings').insert({...})` when the form is submitted. That's it — Supabase handles all the actual database work.

CHAPTER 04

The Admin Dashboard (admin.html)

A private control panel for Dr. Monisha to see every booking, manage their status, and contact patients — all from one screen.

What the Dashboard Shows

When Dr. Monisha opens the admin page, she sees four number cards at the top showing a live count of:

📊 Stats Row

- Total bookings ever received
- Pending (waiting for action)
- Confirmed appointments
- Appointments due today

🔍 Filter & Search

- Search bar (filter by name or phone)
- Tab filters: All / Pending / Confirmed / Rejected
- Sort by newest, oldest, or appointment date
- "Clear Rejected" bulk delete button

Booking Cards — The Main View

Each booking is shown as a card. The left edge of the card is color-coded so Dr. Monisha can instantly see the status at a glance:

Orange = Pending
New booking waiting to be reviewed.

Green = Confirmed
Appointment accepted and confirmed.

Red = Rejected
Booking declined.

Blue = Booked
Appointment fully confirmed and scheduled.

The Confirm Flow — Step by Step

1. **Dr. Monisha clicks "Confirm"**

   A modal dialog opens showing a preview of the booking details.

2. **She optionally types a note**

   For example: "Please bring your previous prescription" or "Join via Google Meet."

3. **She clicks "Confirm Appointment"**

   Two things happen simultaneously: the booking status is updated to "confirmed" in the database, and a WhatsApp message is opened with the patient's number pre-filled.

4. **WhatsApp opens**

   She just taps send — the patient gets a confirmation message immediately.

WhatsApp Integration

WhatsApp doesn't have an official API for sending messages directly. Instead, we use a URL scheme that opens WhatsApp with a pre-written message and a phone number already filled in.

> **ANDROID VS DESKTOP DIFFERENCE**
>
> On Android phones, we use a special `intent://` URL that opens the WhatsApp app directly. On desktop computers, we use the `https://wa.me/` web URL that opens WhatsApp Web. The code detects which device is being used and picks the right URL automatically.

There was a tricky bug here: if we closed the dialog *before* opening WhatsApp, the browser blocked the redirect as a popup. The fix was simple — always call `window.open()` first, then close the dialog.

CHAPTER 05

The Backend — Vercel API Functions

Some operations need to run on a server, not in the browser. For example, we can't expose our secret database key in the public website's code. That's where serverless functions come in.

What is a Serverless Function?

Normally, a "backend" means a server that runs 24/7. Serverless functions are different — they're tiny pieces of code that only run when someone calls them, then shut down. Vercel (our hosting platform) manages all of this automatically. We just write the function; Vercel handles the rest.

Each file inside the `/api/` folder automatically becomes an API endpoint. For example, `/api/get-bookings.js` becomes available at `https://yoursite.vercel.app/api/get-bookings`.

Our Four API Endpoints

FILE	WHAT IT DOES	WHO CALLS IT
`get-bookings.js`	Fetches all bookings from Supabase and returns them as JSON. Uses the secret service key so it bypasses RLS.	Admin dashboard on load
`update-booking.js`	Updates a booking's status (confirm/reject) or deletes it. Also handles bulk-delete of rejected bookings.	Admin dashboard buttons
`save-fcm-token.js`	Saves the doctor's device notification token to Supabase so we know where to send push notifications.	Admin dashboard on load
`send-push.js`	Sends FCM push notifications to all registered devices. Called automatically by Supabase when a new booking is inserted.	Supabase webhook (automatic)

Why We Can't Put the Secret Key in the Website

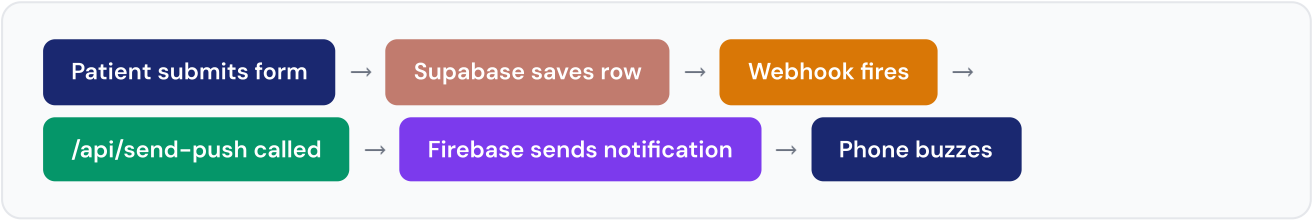
Supabase has two keys: an **anon key** (safe to be public, limited permissions) and a **service role key** (bypasses all security — never expose this publicly). The public website uses the anon key. The API functions use the service key, which is stored in Vercel's environment variables — these are like secret config values that only exist on Vercel's servers, never visible in the code.

ENVIRONMENT VARIABLES

Think of environment variables as a secure notepad on the server. We store secrets like `SUPABASE_SERVICE_KEY` there. The code can read them at runtime, but they're never included in the files that get deployed to GitHub or visible to anyone inspecting the website.

How the Supabase Webhook Works

Supabase has a feature called **webhooks**. We configured it so that every time a new row is inserted into the `bookings` table (i.e., someone submits the booking form), Supabase automatically calls our `/api/send-push` endpoint and sends the new booking data. This is what triggers the push notification to Dr. Monisha's phone.



CHAPTER 06

Push Notifications — Firebase

The goal was simple: the moment a patient submits a booking, Dr. Monisha's phone should buzz — even if she's not on the website.

What is Firebase Cloud Messaging (FCM)?

Firebase is Google's app development platform. One of its services is **Firebase Cloud Messaging (FCM)** — a free system for sending push notifications to phones and browsers. It's the same technology that apps like WhatsApp and Instagram use to send you notifications.

We don't need a native app to use FCM — modern browsers support it through the Web Push standard.

How It Works — The Three Parts

A **Getting Permission + Saving the Token**

When Dr. Monisha first opens the admin dashboard, the browser asks "Allow notifications?". If she clicks Allow, the browser registers with Firebase and receives a unique **FCM token** — like a phone number for that specific device+browser. We save this token to Supabase via `/api/save-fcm-token`.

B **Sending the Notification**

When a new booking comes in, our `/api/send-push` function fetches all saved FCM tokens from Supabase and uses the Firebase Admin SDK to send a message to every token. Firebase handles the actual delivery to each device.

C **Receiving When the Browser is Closed**

This is where the **Service Worker** comes in (covered in the next chapter). A service worker runs in the background even when the browser tab is closed. It listens for incoming FCM messages and displays the system notification.

> **DATA-ONLY MESSAGES**
>
> We send "data-only" FCM messages (no `notification` field). This means the FCM server doesn't automatically show a notification — instead, the message is delivered to our service worker, which then displays the notification itself. This gives us full control over how the notification looks and what happens when it's tapped.

Cleaning Up Old Tokens

FCM tokens expire when a user uninstalls the app, clears browser data, or revokes permission. After sending notifications, our function checks which tokens failed and automatically deletes them from Supabase. This keeps the token list clean and prevents errors.

Testing Notifications

We added a **Test Notification button** in the admin header. Clicking it calls `/api/send-push` directly so Dr. Monisha can verify notifications are working without waiting for a real booking to arrive.

CHAPTER 07

PWA — Installing the App on a Phone

A Progressive Web App (PWA) is a website that can be installed on your home screen and behaves like a native app — no app store required.

What Makes a PWA?

Three ingredients are needed for a website to qualify as an installable PWA:

1. **HTTPS (Secure Connection)**
   The website must be served over HTTPS. Vercel does this automatically — every deployment gets a free SSL certificate.

2. **Web App Manifest (manifest.json)**
   A JSON file that tells the browser how to treat the app — its name, icon, colors, and start page. This is what makes the browser show the "Add to Home Screen" prompt.

3. **Service Worker (sw.js)**
   A JavaScript file that runs in the background, separate from the main page. It handles caching (for offline support) and push notifications.

The Manifest File

Our `manifest.json` defines the app as:

- **Name:** "Dr. Monisha — Dashboard"
- **Start URL:** `/admin.html` (opens the admin panel when launched)
- **Display:** `standalone` — hides the browser's address bar, looks like a real app
- **Theme color:** Navy `#1a2870` — colors the phone's status bar
- **Icons:** 192×192 and 512×512 PNG icons for the home screen

The Service Worker — Two Jobs

Job 1: Caching (Offline Support)

On first visit, the SW downloads and stores key files (HTML, icons) in the browser's cache. If the internet goes down, it serves those cached files instead — the app still loads.

Job 2: Push Notifications

The SW registers with Firebase Messaging and listens for incoming FCM messages using `onBackgroundMessage()`.

Network-first strategy for the admin page: always tries to get the latest version from the internet first, falls back to cache if offline.

Cache-first strategy for images/icons: serves from cache immediately (faster), updates the cache in the background.

When a message arrives (even with the browser closed), the SW displays a system notification using `showNotification()`.

When the notification is tapped, the SW opens `/admin.html`.

Cache Versioning — The "v6" Number

The service worker cache has a name: `dr-monisha-v6`. Every time we deploy changes to the HTML or JavaScript, we bump this number (v6 → v7). This forces the service worker to discard the old cache and download the latest files. Without this, users might see the old version of the site even after we've made updates.

> **WHY PWA INSTEAD OF A NATIVE APP?**
> A native app would require submitting to the App Store (requires a Mac, $99/year, and Apple's review process) or the Play Store (requires a Google account, $25 one-time fee, and review time). A PWA can be installed instantly, updated instantly, and works on every device — all for free.

CHAPTER 08

Deployment — Going Live

Writing the code is one thing. Making it available to the world at a real URL is another. Here's how we deployed everything.

The Deployment Stack

Git — Version Control

All project files are stored in a Git repository. Git tracks every change ever made, so we can always go back to a previous version if something breaks. It also enables the automatic deployment trigger.

Vercel — Hosting & Serverless

Vercel is connected to the Git repository. Every time we push new code, Vercel automatically detects the change and deploys the updated site within seconds. No manual FTP uploads needed.

The Deployment Flow

Write / edit code locally → git commit → git push to GitHub → Vercel auto-deploys → Live in ~30 seconds

Environment Variables in Vercel

Secret keys and credentials are stored in Vercel's dashboard, not in the code. This keeps them safe — even if someone can see the code on GitHub, they can't see the secrets. We set three variables:

VARIABLE	WHAT IT HOLDS
`SUPABASE_URL`	The URL of our Supabase database project
`SUPABASE_SERVICE_KEY`	The secret key that bypasses database security (only used server-side)
`FIREBASE_SERVICE_ACCOUNT`	Firebase credentials for sending push notifications (stored as JSON string)

The Live URL

The site is live at **https://dr-monisha-website.vercel.app**. Vercel also provides a free SSL certificate automatically, so the connection is always encrypted (HTTPS).

Local Development

For testing changes before deploying, we use a local server:

- `node serve.mjs` starts a server at `http://localhost:3000`
- `node screenshot.mjs http://localhost:3000` takes an automated screenshot for visual review
- Once satisfied, we commit and push — Vercel deploys automatically

> **SUMMARY: THE FULL ARCHITECTURE**
>
> The website (HTML/CSS/JS) is hosted on Vercel. Patient bookings go into Supabase (PostgreSQL database). A Supabase webhook calls a Vercel serverless function when a booking is created. That function uses Firebase to send a push notification to the admin dashboard, which is installed as a PWA on Dr. Monisha's phone. The doctor manages bookings from the dashboard and contacts patients via WhatsApp.

REFERENCE

Glossary — Terms Explained Simply

A quick reference for the technical terms used throughout this document.

API

Application Programming Interface. A way for two programs to talk to each other. Like a waiter who takes your order to the kitchen and brings back food — you don't need to know how the kitchen works.

CDN

Content Delivery Network. A network of servers around the world that serve files fast. When we load Tailwind CSS from a CDN, it comes from the nearest server to the user.

DATABASE

An organised way to store data. Like a spreadsheet, but faster, more powerful, and accessible by code from anywhere on the internet.

ENVIRONMENT VARIABLES

Secret configuration values stored on a server, not in code. Like a keychain that only the server can see — the code uses these without exposing them publicly.

FCM TOKEN

A unique identifier Firebase gives to each browser or device that has enabled push notifications. Like a postal address specifically for push notifications.

FIREBASE

Google's app development platform. We use its Cloud Messaging (FCM) service to send push notifications to Dr. Monisha's device.

GIT

A version control system that tracks changes to code over time. Every change is recorded — you can always undo or go back to any previous state.

HTTPS

HyperText Transfer Protocol Secure. A secure version of HTTP. All data sent between the browser and server is encrypted so nobody can intercept it.

PWA

Progressive Web App. A website that can be installed on a phone's home screen and behaves like a native app, including offline support and push notifications.

RLS (ROW LEVEL SECURITY)

A Supabase security feature. By default, the database is locked. You create "policies" to allow specific operations. Without the right policies, reads/writes silently fail.

SERVERLESS FUNCTION

A small piece of backend code that runs only when called (not 24/7). The platform (Vercel) manages all infrastructure — we just write the function.

SERVICE WORKER

A JavaScript file that runs in the background, separate from the web page. It intercepts network requests (for caching) and receives push notifications even when the browser is closed.

SUPABASE

A backend-as-a-service platform. Provides a PostgreSQL database, authentication, and real-time subscriptions — all with a simple API and free tier.

TAILWIND CSS

A CSS framework with thousands of tiny utility classes. Instead of writing CSS files, you apply classes directly in HTML: `text-lg font-bold p-4 rounded-xl`.

VERCEL

A hosting platform for frontend websites and serverless functions. Connects to GitHub and auto-deploys every time code is pushed. Provides free HTTPS and global CDN.

WEBHOOK

An automatic notification from one service to another. Like subscribing to an alert — when something happens in Supabase (new booking), it automatically calls our API endpoint.

The Website is Live

Everything described in this document is running in production. The full stack — public website, admin dashboard, database, push notifications, and PWA — is live and accessible now.

https://dr-monisha-website.vercel.app